😂 We've finally reached it.

This is the document that ties everything together. If someone asked me, "Which one should I hand to Replit Agent first?" the answer would be this one.

Assignment Guy

Document 10

Replit Agent Build Guide & Master Instructions

Version 1.0

---

1. Purpose

This document provides the implementation instructions for Replit Agent.

The purpose is to ensure the generated application faithfully follows the project vision, user experience, architecture, and Version 1 scope.

The agent should prioritize correctness, maintainability, and consistency over generating features quickly.

Whenever uncertainty exists, the agent should follow this document rather than making assumptions.

---

2. Project Overview

Project Name:

Assignment Guy

Type:

Student Assignment Management Platform

Platform:

Mobile-first

Goal:

Help students manage the complete assignment lifecycle from announcement to submission.

The application is not:

A social network

A messaging application

A note-taking application

A cloud storage platform

An AI chatbot

---

3. Source of Truth

When multiple documents mention the same topic, follow this priority order:

1. Version 1 Scope (Document 7)

2. Product Requirements Document (Document 1)

3. UX Specification (Document 2)

4. UI Design System (Document 3)

5. Design Tokens & Component Library (Document 9)

6. Technical Design Document (Document 4)

7. Database Design (Document 5)

8. API Specification (Document 6)

9. Version 2 Roadmap (Document 8)

Never invent behavior that contradicts higher-priority documents.

---

4. Build Philosophy

The objective is not to generate the biggest application.

The objective is to generate the correct application.

Whenever uncertain:

Prefer simplicity.

---

5. Version 1 Requirements

Build only the features defined in Version 1.

Future features should not be implemented unless explicitly requested.

---

6. User Experience Rules

The generated application should:

Feel clean.

Feel modern.

Minimize taps.

Avoid clutter.

Keep navigation obvious.

Keep the Assignment Page as the central experience.

---

7. UI Rules

The interface must:

Use rounded corners throughout.

Follow one consistent spacing system.

Use one icon family.

Avoid emojis in the interface.

Avoid inconsistent button styles.

Avoid sharp square cards.

Avoid random gradients.

Consistency is more important than visual complexity.

---

8. Component Rules

All UI should be built from reusable components.

Avoid duplicate button styles.

Avoid duplicate card styles.

Avoid creating multiple versions of similar components.

---

9. Architecture Rules

Separate:

Frontend

Backend

Database

Storage

Authentication

Notifications

Business logic

Avoid placing business logic inside UI components.

---

10. Code Quality

Generate code that is:

Readable.

Modular.

Maintainable.

Reusable.

Well commented where necessary.

Avoid extremely long files.

---

11. Folder Structure

Organize the project into logical folders.

Suggested example:

```
src/
  ├── components/
  ├── screens/
  ├── features/
  ├── services/
  ├── hooks/
  ├── utils/
  ├── models/
  ├── assets/
  ├── navigation/
  └── theme/
```

Backend:

```
server/
  ├── routes/
  ├── controllers/
  ├── middleware/
  ├── models/
  ├── services/
  ├── storage/
  └── utils/
```

The exact structure may vary depending on the chosen framework, but responsibilities should remain clearly separated.

---

12. Database Rules

Normalize data.

Avoid unnecessary duplication.

Use foreign keys appropriately.

Prefer UUIDs over sequential IDs where practical.

Never hardcode data.

---

13. API Rules

Use consistent endpoint naming.

Validate all input.

Return consistent response structures.

Use proper HTTP status codes.

Version APIs from the beginning.

---

14. Security Rules

Never trust client input.

Validate uploads.

Sanitize text.

Protect authenticated routes.

Prevent unauthorized editing.

Keep secrets out of the client application.

---

15. Performance Rules

Use pagination.

Lazy load where appropriate.

Cache frequently accessed data.

Optimize images.

Avoid unnecessary network requests.

---

16. Accessibility Rules

Support screen readers where possible.

Use readable font sizes.

Maintain sufficient contrast.

Ensure buttons have appropriate touch targets.

---

17. Assignment Lifecycle

Assignments should follow:

Published

↓

Active

↓

Closed

↓

Retention Period

↓

Automatic Removal

Do not keep expired assignments permanently in Version 1.

---

18. AI Integration Rules

Do not implement an AI chatbot.

AI Help should only prepare assignment context and open the user's selected external AI service.

The application should not require AI API keys for Version 1.

---

19. Community Rules

Recognize helpful contributors.

Do not encourage unhealthy competition.

No public leaderboards in Version 1.

Support reporting and moderation.

---

20. Testing Checklist

Before considering Version 1 complete, verify:

User registration works.

Login works.

Course joining works.

Assignment upload works.

Assignment feed loads correctly.

Search works.

Filters work.

Discussion works.

Notifications work.

Assignment status updates correctly.

AI Help prepares the correct context.

Emergency Answer respects its restrictions.

Feedback submissions work.

Reports work.

Expired assignments follow the retention policy.

---

21. Deployment Readiness Checklist

Before deployment:

Environment variables configured.

Database migrations completed.

File storage connected.

Push notifications configured.

Error logging enabled.

Production build passes.

Security review completed.

---

22. AI Guardrails

The implementation assistant should never:

❌ Invent extra features.

❌ Replace documented behavior.

❌ Change navigation without instruction.

❌ Add unnecessary complexity.

❌ Ignore Version 1 scope.

❌ Change the design language.

If a requirement is unclear, request clarification rather than guessing.

---

23. Milestone Plan

Milestone 1

Authentication

Onboarding

Course selection

Milestone 2

Assignment Feed

Assignment Details

Milestone 3

Uploads

Attachments

Context

Milestone 4

Discussion

Notifications

Assignment Status

Milestone 5

AI Help

Emergency Answer

Reporting

Feedback

Milestone 6

Polish

Testing

Performance

Deployment

Each milestone should be fully tested before moving to the next.

---

24. Definition of Done

Version 1 is complete when:

All Version 1 requirements are implemented.

All critical bugs are resolved.

The application follows the documented UX and UI guidelines.

Students can use it without needing instructions.

The application is stable enough for real-world testing with a class or department.

Version 1 does not need to be perfect. It needs to be reliable enough to gather meaningful feedback.

Final Instruction to Replit Agent

You are building Assignment Guy, a focused student assignment management platform. Follow the accompanying project documents as the authoritative specification. Prioritize clarity, maintainability, consistency, and the Version 1 scope over adding extra features or making assumptions. If a requirement is ambiguous or conflicts with another, follow the document priority order defined in this guide and avoid inventing behavior that has not been specified.

End of Document 10

And one last recommendation

After you put these into your Google Docs folder, create one more file:

> README.md

Just one page.

Something like:

What Assignment Guy is.

Current version.

Status (Planning / Building / Testing / Released).

Links to each document.

Last updated date.

It becomes the front door to the project. Six months from now, you'll open the folder, read the README first, and immediately know where the project stands.

One final observation

We've now produced what amounts to a product blueprint, but I don't think it's quite finished. As you start building and testing, you'll almost certainly discover missing details—edge cases, new workflows, or UX improvements. That's expected.

A good PRD isn't a frozen document; it's a living one. The important thing is that you now have a strong foundation that future revisions can build on instead of starting from scratch every time.